

FlowSync — Integrated Order, Inventory & Warehouse Management System

Project Documentation

1. Introduction / はじめに

English:

FlowSync is a full-stack business web application built to manage the complete lifecycle of customer orders, product inventory, and warehouse fulfillment on a single platform. It connects the customer-facing ordering process with backend inventory control and warehouse status tracking, so that stock levels stay accurate as orders are placed and fulfilled.

日本語:

FlowSync は、顧客注文・商品在庫・倉庫出荷業務のライフサイクル全体を一つのプラットフォームで管理するために開発されたフルスタック業務アプリケーションです。顧客側の注文プロセスと、バックエンドの在庫管理・倉庫ステータス追跡を連携させることで、注文の発生と処理に応じて在庫数量を正確に保ちます。

2. Project Overview / プロジェクト概要

English:

The platform allows customers to browse an active product catalog, place orders, and track order status through to delivery. Every order is validated and processed on the server: stock availability is checked, inventory is deducted transactionally, and the order total is calculated from server-side product data rather than client input. Administrators and warehouse managers manage the product catalog, monitor multi-warehouse inventory, execute inter-warehouse stock transfers, audit operational actions, and move orders through a defined fulfillment and warehouse pick/pack workflow.

日本語:

本プラットフォームでは、顧客が有効な商品カタログを閲覧し、注文を行い、配送までの注文ステータスを追跡できます。すべての注文はサーバー側で検証・処理されます。在庫の有無を確認し、トランザクション処理により在庫を減算し、注文合計はクライアントからの入力ではなくサーバー側の商品データに基づいて算出されます。管理者および倉庫担当者は商品カタログの管理、複数倉庫にまたがる在庫の監視、倉庫間在庫移動の実行、業務操作の監査ログ確認、および定義された出荷・倉庫ピッキング/梱包ワークフローに沿った注文の処理を行います。

3. Problem Statement / 課題

English:

Businesses handling both online orders and physical inventory often struggle to keep stock counts accurate in real time, leading to overselling, delayed fulfillment, and manual reconciliation between order systems and warehouse floors. FlowSync addresses this by tying order creation and inventory deduction together in a single atomic transaction with database-level locking, providing multi-warehouse stock transfer state management, maintaining an immutable audit log of all critical changes, and giving staff a clear, status-driven view of where each order stands from placement to dispatch.

日本語:

オンライン注文と実在庫の両方を扱う事業者は、リアルタイムでの在庫数の正確な把握に苦勞することが多く、売り越しや出荷遅延、注文システムと倉庫現場との手作業による突合が発生しがちです。FlowSync は、注文作成と在庫減算をデータベースロックを伴う一つの原子的な処理として結び付け、複数倉庫間の在庫移動ステータス管理、すべての重要操作に対する不変の監査ログ、および注文が受付から出荷までどの段階にあるかをステータスベースで明確に確認できる仕組みを提供することで、この課題に対応します。

4. Objectives / 目的

- **English:** Provide accurate, real-time inventory tracking tied to order processing
日本語: 注文処理と連動したリアルタイムかつ正確な在庫管理を実現する
- **English:** Ensure transactional integrity and prevent overselling via pessimistic and optimistic locking
日本語: 悲観的・楽観的ロックにより、同時注文時の売り越しを防止しトランザクションの整合性を確保する
- **English:** Support multi-warehouse logistics with structured stock transfer requests and approvals
日本語: 承認・出荷・入荷確認を伴う構造化された複数倉庫間在庫移動ワークフローを提供する
- **English:** Guarantee enterprise traceability and compliance through immutable audit logging
日本語: 不変の監査ログにより、在庫変更や業務操作の追跡可能性とコンプライアンスを担保する
- **English:** Give warehouse staff a clear order and pick/pack status workflow
日本語: 倉庫スタッフに明確な注文・ピッキング/梱包ステータスのワークフローを提供する
- **English:** Support secure, role-based access control (RBAC) across administrative and operational roles
日本語: 顧客、管理者、倉庫管理者、営業に応じた安全なロールベースアクセス制御（RBAC）を実現する
- **English:** Design the system to be cloud-deployable, containerized, and horizontally scalable
日本語: コンテナ化され、クラウド上へのデプロイと水平スケーリングが可能なシステム設計とする

5. Key Features / 主な機能

- **English:** Fine-grained Role-Based Access Control (RBAC) with ADMIN, WAREHOUSE_MANAGER, SALES, and CUSTOMER roles
日本語: ADMIN、WAREHOUSE_MANAGER、SALES、CUSTOMER ロールに対応したきめ細やかなアクセス制御
- **English:** Multi-Warehouse Stock Transfer System with a 5-stage lifecycle (PENDING → APPROVED → IN_TRANSIT → COMPLETED / CANCELLED)
日本語: 5段階のステータス遷移 (PENDING → APPROVED → IN_TRANSIT → COMPLETED / CANCELLED) と自動在庫相殺・復元ロジックを備えた複数倉庫間在庫移動機能
- **English:** Immutable Audit Logging tracking actor, action, target entity, previous value, new value, and timestamp
日本語: 操作者、アクション、対象エンティティ、変更前の値、変更後の値、タイムスタンプを永続記録する不変の監査ログ機能
- **English:** High-concurrency transaction management with pessimistic write locking (SELECT ... FOR UPDATE) and optimistic locking (@Version)
日本語: 同時注文による売り越しを防ぐ悲観的ロック (SELECT ... FOR UPDATE) と在庫更新の楽観的ロック (@Version)
- **English:** Product catalog with real-time stock levels, warehouse locations, and low-stock threshold monitoring
日本語: リアルタイムの在庫数、倉庫ロケーション情報、発注点管理を含む商品カタログ
- **English:** Transactional order placement with server-side price calculation and stock validation
日本語: サーバー側での価格計算と在庫検証を伴うトランザクション処理による注文登録
- **English:** Order status (PLACED → CONFIRMED → PROCESSING → SHIPPED → DELIVERED) and pick status (PENDING_PICK → PICKED → PACKED → SHIPPED) tracking
日本語: 注文ステータスおよびピッキング/梱包ステータスの進捗追跡
- **English:** Unified Admin & Operations dashboard with order, inventory, and low-stock overview
日本語: 注文・在庫・低在庫状況を一元的に把握できる管理者・運用ダッシュボード
- **English:** Scheduled background job (@Scheduled) to detect and log low-stock products dipping below reorder level
日本語: 低在庫商品を検知しログに記録する定期バックグラウンドジョブ
- **English:** Centralized exception handling returning sanitized, structured JSON error responses
日本語: 整形されたエラーレスポンスを返す一元化された例外処理

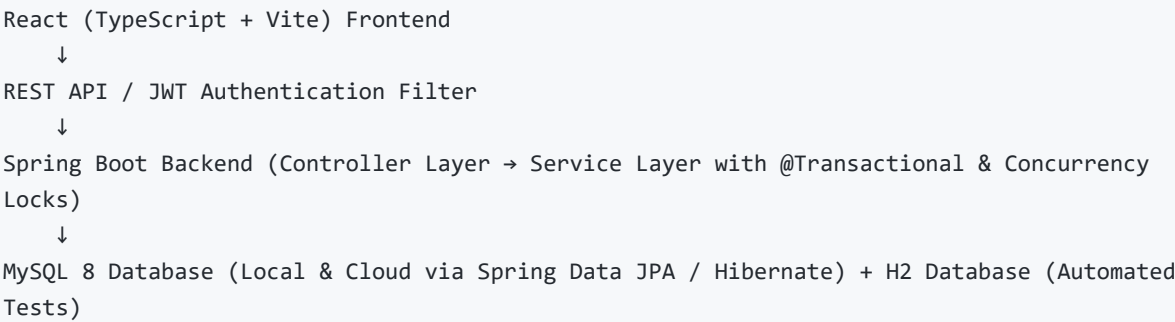
6. System Architecture / システム構成

English:

The frontend (React + TypeScript + Vite) communicates with the Spring Boot backend over REST APIs secured by stateless JWT authentication. The backend is organized into Controller, Service, Repository, and Entity layers, with **MySQL 8** as the relational data store and an in-memory H2 database for automated test suite execution.

日本語:

フロントエンド（React + TypeScript + Vite）は、ステートレスな JWT 認証で保護された REST API を介して Spring Boot バックエンドと通信します。バックエンドは Controller・Service・Repository・Entity の各層に整理されており、リレーショナルデータストアとして **MySQL 8** を使用し、自動テストには高速な H2 インメモリデータベースを活用しています。



7. Technology Stack / 技術スタック

Category	Technologies Used
Frontend	React 18 / 19, TypeScript, Vite, React Router, Axios
Backend	Java 21, Spring Boot 3.3.x, Spring Data JPA, Spring Security
Database	MySQL 8 (via mysql-connector-j and Hibernate JPA), H2 (In-memory testing)
Concurrency & Locking	Pessimistic Write Locking (PESSIMISTIC_WRITE), Optimistic Locking (@Version)
Authentication & RBAC	Stateless JWT (jjwt 0.12.x), BCrypt, Spring Security @PreAuthorize
Testing Suite	JUnit 5, Mockito, Spring Boot Test, MockMvc (18 automated tests)
Containerization	Docker, Docker Compose (MySQL 8 + Backend + Frontend multi-container)
Current Deployment	Backend on Render / Docker; Frontend on Vercel
Cloud Architecture (Planned)	AWS EC2 (backend hosting), AWS RDS (MySQL 8 database), AWS S3 + CloudFront (frontend distribution) — designed and documented
Tools	Git, GitHub, Maven, npm, Postman

8. Current Status & Roadmap / 現在の状況と今後の計画

English:

The core order, inventory, multi-warehouse stock transfer, audit logging, and RBAC authentication flows are fully implemented, verified, and containerized with Docker Compose. A suite of 18 automated unit and integration tests verifies business logic, concurrency safety, and role enforcement. AWS cloud migration (EC2, RDS MySQL, S3/CloudFront) is designed and documented, ready for execution as the production infrastructure phase.

日本語:

注文・在庫・複数倉庫間在庫移動・監査ログ・RBAC認証の中核フローは完全に実装され、Docker Compose によるコンテナ化および動作検証が完了しています。18件の自動単体・統合テストにより業務ロジック、並行性安全性、ロール権限が厳格に検証されています。AWS クラウドへの移行（EC2、RDS MySQL、S3/CloudFront）は設計・文書化済みであり、本番インフラフェーズとして展開可能な状態です。

9. Conclusion / 結論

English:

FlowSync demonstrates practical, enterprise-grade application of Java/Spring Boot backend development, relational database design with MySQL 8, transactional data integrity, pessimistic locking against concurrency race conditions, inter-warehouse logistics state machines, immutable audit compliance, and React/TypeScript frontend development to a real business problem — connecting order processing with accurate, real-time multi-warehouse inventory and logistics operations.

日本語:

FlowSync は、Java/Spring Boot によるバックエンド開発、MySQL 8 によるリレーショナルデータベース設計、トランザクションデータの完全性、同時実行競合を防ぐ悲観的ロック、倉庫間物流ステータスマシン、不変の監査コンプライアンス、および React/TypeScript によるフロントエンド開発を、注文処理と正確でリアルタイムな複数倉庫在庫・物流業務という実際の業務課題に実践的かつエンタープライズ水準で適用したものです。

GitHub Link: <https://github.com/KvPradeepthi/flowsync>

10. Developer Information / 開発者情報

Veera Pradeepthi Kamichetty

Full-Stack Developer | Java / Spring Boot | React / TypeScript | MySQL | B.Tech, Aditya University